CSE1005 — PYTHON PROGRAMMING

Module 2

Control Structures & Functions

Lab Notes & Exercise Sheet — Conditional Statements, Loops, Loop Control, Functions, Lambda Functions

D Y Patil International University — School of Computer Science, Engineering & Applications

1. Conditional Statements

1.1 if Statement

Use if when a block of code should run only when a condition is true.

```
if condition:
    statement
```

Example — check whether a number is positive:

```
n = 10
if n > 0:
    print("Positive number")
```

Python checks the condition. If it is True, the indented code runs. If it is False, Python skips it.

1.2 if-else

Use if-else when there are two possible paths.

```
age = 20
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

1.3 if-elif-else

Use elif when there are more than two possible conditions.

```
marks = 72
if marks >= 75:
    print("Distinction")
elif marks >= 60:
    print("First Class")
elif marks >= 40:
    print("Pass")
else:
    print("Fail")
```

Conditions are checked from top to bottom. Once one condition is true, its block runs and the remaining elif blocks are skipped.

1.4 Comparison & Logical Operators

These operators are used to build the conditions inside if statements.

Operator	Meaning	Example
`==`	equal to	`x == 10`
`!=`	not equal to	`x != 10`
`>`	greater than	`x > 10`
`<`	less than	`x < 10`
`>=`	greater than or equal	`x >= 10`
`<=`	less than or equal	`x <= 10`
`and`	true only if both conditions are true	`age>=18 and has_id`
`or`	true if at least one condition is true	`day=='Sat' or day=='Sun'`
`not`	reverses True/False	`not is_raining`

1.5 Nested if

A nested if is an if statement written inside another if statement.

```
age = 20
has_id = True
if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Entry not allowed")
```

Make sure students are comfortable with plain if-else and indentation before introducing nested if.

2. Loops

A loop repeats a block of code, so you don't have to write the same statement again and again — for example, printing something 100 times.

2.1 for Loop

Use a for loop to iterate over a sequence or a known range of values.

```
for i in range(1, 6):
    print(i)
```

Output: 1 2 3 4 5

2.2 range()

range() is the most commonly used helper with for loops.

Expression	Values Produced
`range(5)`	`0, 1, 2, 3, 4`
`range(1, 6)`	`1, 2, 3, 4, 5`
`range(2, 11, 2)`	`2, 4, 6, 8, 10`
`range(5, 0, -1)`	`5, 4, 3, 2, 1`

range(start, stop) includes start but excludes stop — so range(1, 5) gives 1, 2, 3, 4.

2.3 More for Loop Examples

Print even numbers from 2 to 10:

```
for i in range(2, 11, 2):
    print(i)
```

Multiplication table:

```
n = 5
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Sum of numbers from 1 to 10:

```
total = 0
for i in range(1, 11):
    total = total + i
print("Sum =", total)
```

2.4 while Loop

A while loop repeats as long as its condition stays true.

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

Watch Out For This

Something inside the loop must eventually make the condition false, otherwise the loop keeps running forever — this is called an infinite loop.

2.5 for vs while

for	while
Good when the number of repetitions is known	Good when repetition depends on a condition
Often used with range()	You usually update a variable yourself inside the loop
Example: print numbers 1 to 10	Example: keep asking the user for input until they enter 0

3. Loop Control Statements

3.1 break — Stop the Loop

break immediately exits the nearest loop.

```
for i in range(1, 11):
    if i == 5:
        break
    print(i)
```

Output: 1, 2, 3, 4

3.2 continue — Skip the Current Iteration

continue skips the rest of the current iteration and moves to the next one.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Output: 1, 2, 4, 5

3.3 pass — Do Nothing

pass is a placeholder. It lets you write an empty block without causing a syntax error — useful when you're still deciding what code should go there.

```
for i in range(5):
    pass
print("Program continues")
```

Quick Way to Remember

break = stop the loop, continue = skip this round, pass = do nothing for now.

4. Functions

4.1 Why Functions?

A function is a reusable block of code that performs a specific task. Instead of writing the same logic again and again, you define it once and call it whenever you need it — like a small machine: input goes in, processing happens, output comes out.

```
def greet():
    print("Hello!")

greet()
greet()
```

4.2 Function Syntax

```
def function_name(parameters):
    # function body
    statements
    return value
```

- def starts the function definition.
- parameters are the inputs the function receives.
- return sends a result back to the code that called the function.

4.3 Function with Parameters

```
def add(a, b):
    print(a + b)

add(10, 20)
add(5, 7)
```

Here a and b are **parameters** — the names used inside the function. The values 10 and 20 are the **arguments** — the actual values passed when the function is called.

4.4 Function with return

Use return when the function should give a value back to the caller.

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)
```

print()	Displays a value on the screen.
return	Sends a value back so the caller can store or use it further.

4.5 Practical Function Examples

Square of a number:

```
def square(n):
    return n * n

print(square(5))
```

Check even or odd:

```
def is_even(n):
    return n % 2 == 0

print(is_even(10))
print(is_even(7))
```

Factorial using a function:

```
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result = result * i
    return result

print(factorial(5))
```

4.6 Default Parameters

A parameter can have a default value, used when no argument is supplied for it.

```
def greet(name="Student"):
    print("Hello", name)

greet()
greet("Rahul")
```

5. Lambda Functions

A lambda is a small, anonymous (unnamed) function. It is useful when the function is short and contains just a single expression.

```
square = lambda x: x * x
print(square(5))
```

This is equivalent to the normal function:

```
def square(x):
    return x * x
```

For now, focus on understanding the syntax through simple one-line examples like this — lambda is used more heavily later, alongside functions like map() and filter().

6. Worked Examples

Example 1 — Positive, Negative or Zero

```
n = int(input("Enter a number: "))
if n > 0:
    print("Positive")
elif n < 0:
    print("Negative")
else:
    print("Zero")
```

Example 2 — Sum of Even Numbers up to N

```
n = int(input("Enter N: "))
total = 0
for i in range(2, n + 1, 2):
    total += i
print("Sum =", total)
```

Example 3 — Menu using while + break

```
while True:
    print("1. Say Hello")
    print("2. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        print("Hello!")
    elif choice == 2:
        break
    else:
        print("Invalid choice")
```

Example 4 — Function + Condition

```
def grade(marks):
    if marks >= 75:
        return "Distinction"
    elif marks >= 60:
        return "First Class"
    elif marks >= 40:
        return "Pass"
    else:
        return "Fail"

m = int(input("Enter marks: "))
print(grade(m))
```

7. Common Mistakes to Watch For

Mistake	Explanation
Missing colon (:)	if, elif, else, for, while and def all need a colon at the end of the line.
Wrong indentation	All statements inside the same block must be indented consistently.
Using = instead of ==	= assigns a value; == compares two values.
Infinite while loop	Check whether the loop variable actually changes toward the stopping condition.
range(1, 5) gives only 1–4	The stop value is always excluded from the result.
Confusing print and return	print() only displays a value; return sends it back to the caller to use.
Calling a function before defining it	Python reads top to bottom, so define the function above the line that calls it.

Module 2 — Cheat Sheet

Keyword	Purpose	Mini Syntax
`if`	Run a block if a condition is true	`if condition:`
`elif`	Try another condition	`elif condition:`
`else`	Fallback block	`else:`
`for`	Repeat over a sequence/range	`for i in range(...):`
`while`	Repeat while a condition is true	`while condition:`
`break`	Stop the loop	`break`
`continue`	Skip the current iteration	`continue`
`pass`	Placeholder / do nothing	`pass`
`def`	Define a function	`def name(...):`
`return`	Send a value back	`return value`
`lambda`	Short anonymous function	`lambda x: expression`

Revision Questions

Go through these before the next lab — you should be able to answer each one in a line or two.

- What is the difference between if and if-else?
- Why is indentation important in Python?
- What is the difference between for and while?
- What does range(5) produce?
- What is the purpose of break?
- What is the difference between break and continue?
- Why do we use pass?
- What is a function?
- What is the difference between a parameter and an argument?
- Why do we use return?
- What is a lambda function?

Lab Exercise Sheet — Module 2

Attempt every question yourself first. Save each program as a separate .py file and run it to check the output before moving to the next question.

Section A — Conditional Statements

1. Write a program to check whether a number entered by the user is even or odd.
2. Write a program to find the largest of two numbers entered by the user.
3. Write a program that takes a person's age and prints “Child”, “Teenager” or “Adult” using if-elif-else.
4. Write a program that checks whether a year is a leap year (Hint: use the and operator with the conditions `year % 4 == 0` and `year % 100 != 0`, or divisible by 400).

Section B — Loops

1. Print numbers from 1 to 20 using a for loop.
2. Print all the even numbers between 1 and 50 using a for loop.
3. Print the multiplication table of any number entered by the user.
4. Find the sum of numbers from 1 to N, where N is entered by the user.
5. Print numbers from 10 down to 1 using a while loop.

Section C — Loop Control

1. Print numbers from 1 to 20, but stop the loop as soon as you reach 15 (use break).
2. Print numbers from 1 to 10, but skip the number 6 (use continue).
3. Write a program that keeps asking the user to enter a number, and stops when the user enters 0 (use while and break).

Section D — Functions

1. Write a function called square that takes a number and returns its square. Call it with three different numbers.
2. Write a function called is_even that takes a number and returns True or False.
3. Write a function called factorial that takes a number and returns its factorial using a loop.
4. Write a function called largest that takes three numbers and returns the largest of them.

Section E — Lambda & Mixed Practice

1. Write a lambda function that takes a number and returns its cube. Test it with two different numbers.
2. Write a program that finds the Fibonacci series up to N terms using a for or while loop.
3. Write a program that checks whether a number entered by the user is prime.

Bonus (Optional — for those who finish early)

Build a simple menu-driven calculator: show options for Add, Subtract, Multiply, Divide and Exit using a while loop, and use a separate function for each operation.

Submission Note

Save each program as a separate .py file named with the question number (e.g. A2.py, D1.py) in a folder named with your roll number, and be ready to demonstrate the output on your machine.